

macOS Server Reverse Proxy 2.0.6

In some smaller organizations, it may be necessary to run more than one server on the same computer or run more than one server behind a single WAN i.p. address. This guide can help you run multiple http server apps/appliances like FileMaker Server, Kerio Connect, Maxum Rumpus and Synology NAS on the same computer/network as a functioning macOS Server.* It does so using Apache reverse proxies implemented using Apple's recommended technique of web apps to survive updates to macOS Server.* In the case of Synology it can reverse proxy from the standard port 443 to Synology's SSL port of 5001 - which can be useful for security and availability.

***Not saying it is "Best Practices", nor that we recommend it; just saying you can do it.**

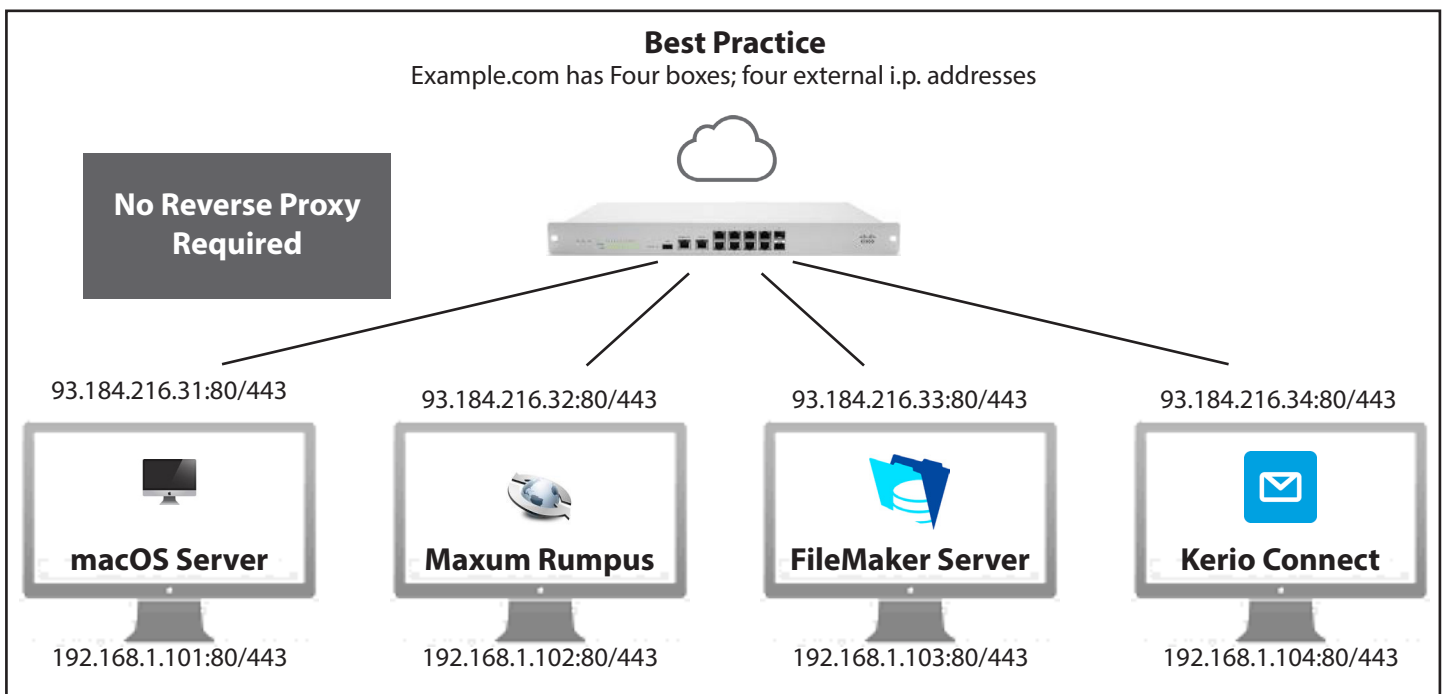


Fig 1. DNS In Best Practice the enterprise level router uses four external i.p. addresses for four internal boxes. Each can have its own port 80 and port 443 traffic. Sweet!

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

i.p. address scarcity

Example.com has four (4) boxes; but only one (1) external i.p. address

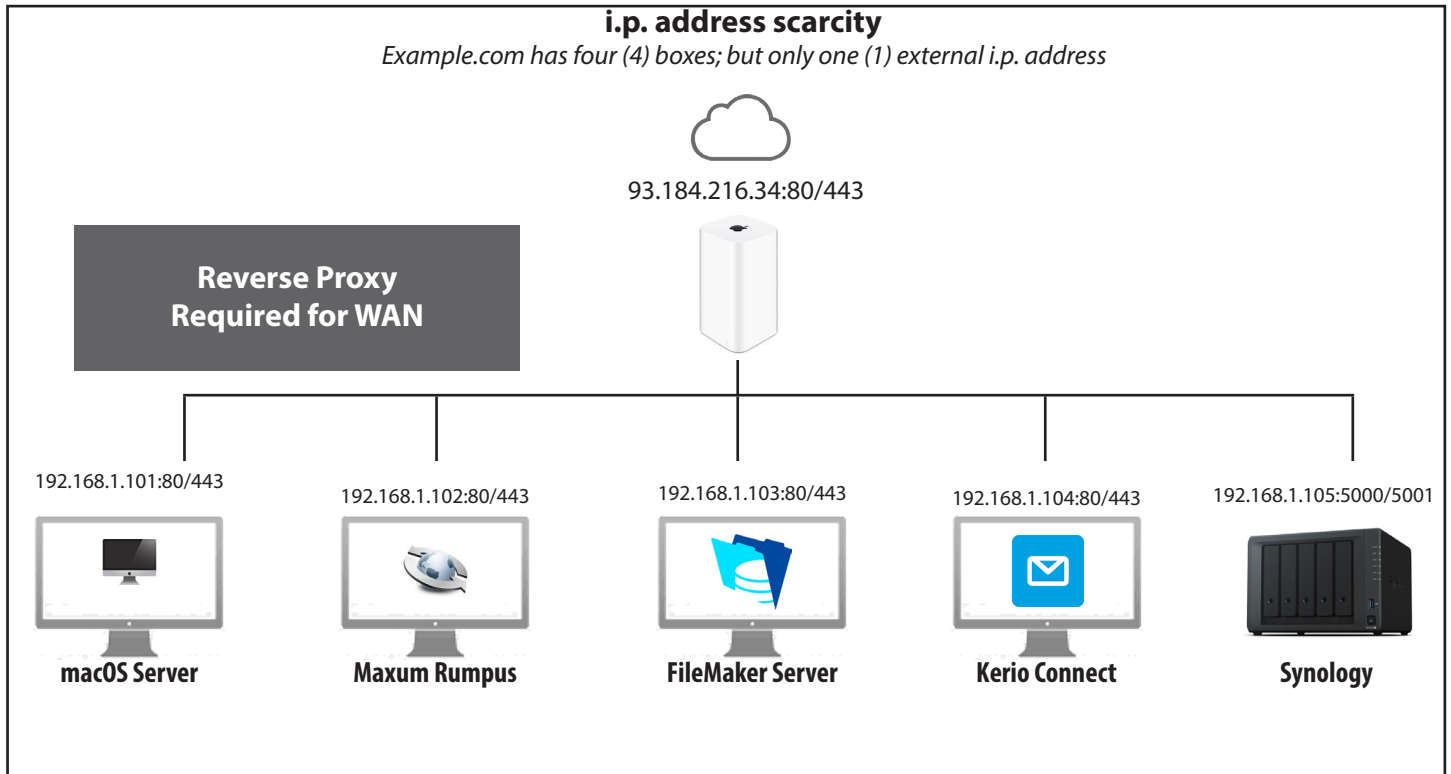


Fig. 2 There is only one external i.p. address available. The AirPort Extreme router can only pass port 80 and 443 to one of the internal boxes. You use reverse proxy so that DNS calls for Rumpus, FileMaker, and Kerio send port 80 and 443 calls to the correct internal i.p. address.

hardware and i.p. address scarcity

Example.com has only one (1) boxes and only one (1) external i.p. address

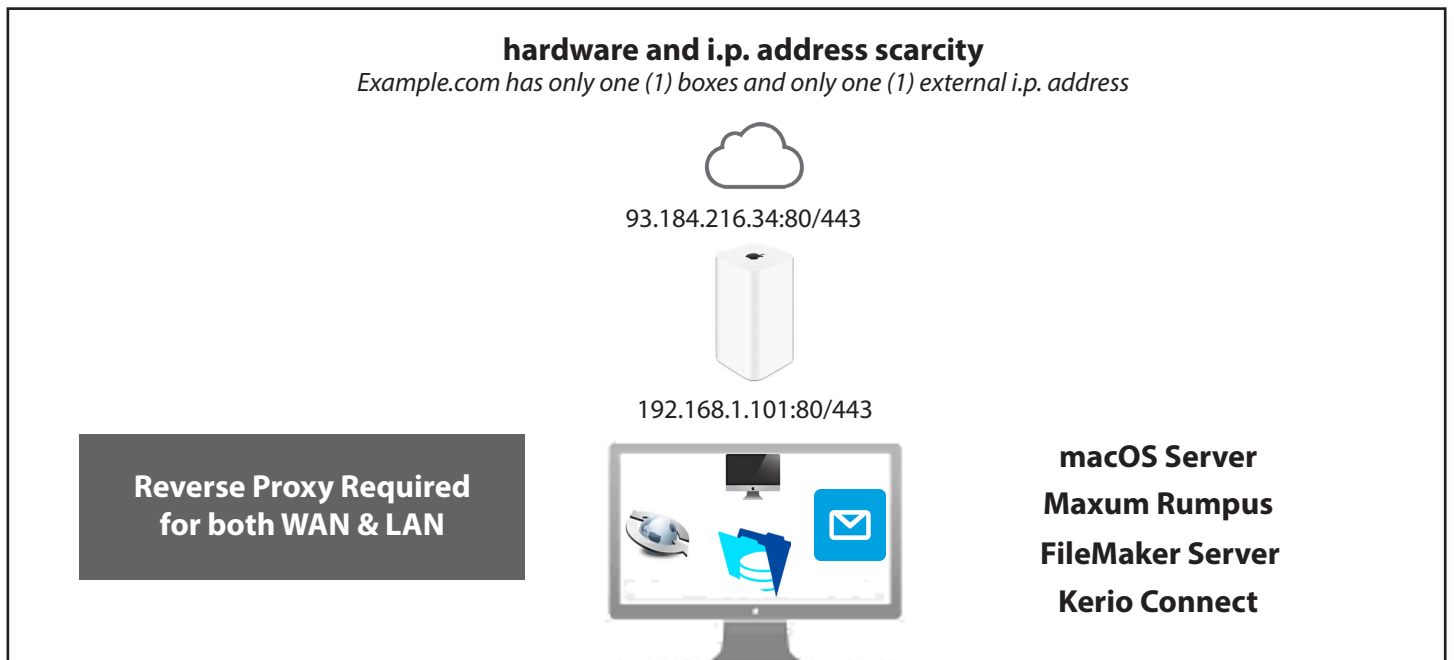


Fig. 3 There is only one external i.p. address and only one server. The AirPort Extreme router passes port 80 and 443 to the server and you use reverse proxy so that DNS calls for Rumpus, FileMaker, and Kerio send port 80 and 443 calls to the correct service running on altered ports.

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

This version of the tutorial (2.0.3) has been tested on macOS High Sierra 10.13/macOS Server 5.4 and on macOS Sierra 10.12/macOS Server 5.3.1.

macOS Server 5 introduced reverse proxy to EVERY custom web site and security was tightened in 5.2 leading to some modification of my scripts. For Yosemite (10.10) and OS X Server 4 or earlier please consult version 1.0.10 of this tutorial which is appended at the end for nostalgia – see Appendix E.

So, we are going to:

- update to the latest macOS (10.13.0 at the time of this writing)
- install macOS Server and provide DNS
- set up Open Directory
- set up our Reverse Proxies
- install Rumpus, FileMaker Server and Kerio Connect, and Rumpus
- configure them to other ports
- create custom web sites that use our special Reverse Proxy web apps
- fix a Rumpus for SSL Proxy and Kerio for Proxy Timeout issues.

First lets get our services, dns and ports sorted out (fig. 4):

Server	FQDN	HTTP	HTTPS
macOS Server	macOSServer.precursor.ca	80	443
Maxum Rumpus	rumpus.precursor.ca	8000	8010
FileMaker Server	filemaker.precursor.ca	8001	8011
Kerio Connect	kerio.precursor.ca	8003	8013
Synology	synology.precursor.ca	5000	5001

Fig 4. DNS and port assumptions

1. Purchase and install macOS Server from the App Store. (For macOS High Sierra use v.5.4 or higher).
2. Set up DNS. I am going to assume you know what you are doing here as it is beyond the scope of this tutorial to deal with this. You should have an A record for the Fully Qualified Domain Name (FQDN) of your server: *osxserver.precursor.ca*
And you should have aliases (CNAMEs) for Rumpus, Filemaker Server, and Kerio Connect (as per figure 4 above).
3. Set up an Open Directory but turning it on for the FQDN of your server: *osxserver.precursor.ca*
4. Turn on Web Services in macOS Server. In previous versions of the Tutorial and the Server app we had to turn Apache on and off. That is no longer required.
6. Now we are going to install our 3rd party servers.

Rumpus

Lets start with Maxum Rumpus as an example since it defaults to port 8000 for web.

Download Rumpus from www.maxum.com and place it in your Applications folder.

Launch Rumpus and in its SetUp Assistant step 3 Enable the WFM for port 8000:

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

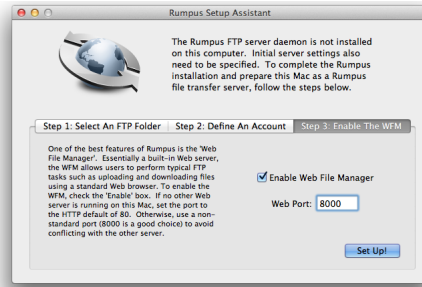


Fig 5. Enable WFM on port 8000 in Rumpus

You may also want to enable secure web services for Rumpus. This is done in the Network Settings: Secure Services tab. For this example install we chose port 8010 as the HTTPS port.

You will also want to tell Rumpus that it is going to be running behind a reverse proxy. Do so by choosing "Open Config Folder" from the "File" menu. Then modify the "Rumpus.conf" file to include the line:

```
WFMThruProxy YES
```

If you are also going to be using Rumpus with SSL then you will need to add this to the "Rumpus.conf file":

```
HTTPTThruProxy YES
```

Then save the file, and stop and restart Rumpus.

We assume you can deal with certificates here either self-signed or commercial – In Appendix A you will some info about moving certificates between OS X and these apps.

FileMaker Server 16*

That was easy. Now let's do FileMaker Server. (If you are using an earlier version then you need to do some more finessing and you will want to look at version 1.0.10 of this Tutorial in the Appendix E).

FileMaker Server 16 won't install if macOS Server is running. So before running the installer, move the Server.app from the Applications folder to the Trash. This will cause the server processes to stop as the system detects that Server has been removed. BUT the server configuration is kept in /Library/Server and this will remain untouched! So, when you are finished the FileMaker Server installation just move Server.app back into the Applications folder and launch it to restart all the services you already configured.

During the installation of FileMaker Server 16 when asked, change the standard HTTP and HTTPS ports to 8001 and 8011 respectively.

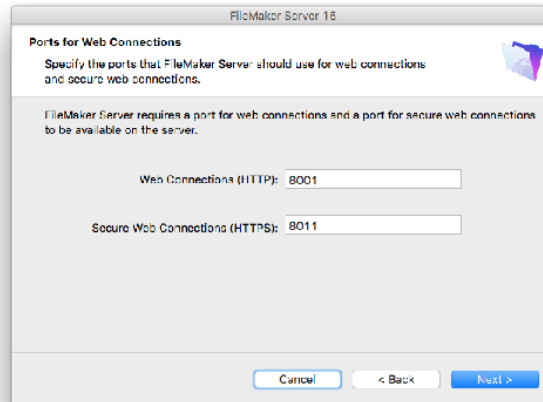


Fig 6. Altering the web ports during FileMaker Server installation

NB. The FileMaker Server Admin Console name ("server Name" in the General Settings tab of Admin Console)

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

must be identical to the Apple System Preferences: Sharing name. (FileMaker Answer ID 14127).

Kerio Connect

Download Kerio Connect and the Kerio Connect Open Directory Plugin.

Install the Open Directory Plugin first. The install Install Kerio Connect (note you need the Java JDK and not the Java JRE which got installed with FileMaker. You may also use Apple's OS X Java for Mac 2015).

Since OS X Server 5 even only running OD will be listening on ports 80, 443, 8008, 8800 8443 and 8843. We will reset Kerio's HTTP and HTTPS services to use ports 8003 and 8013 respectively.

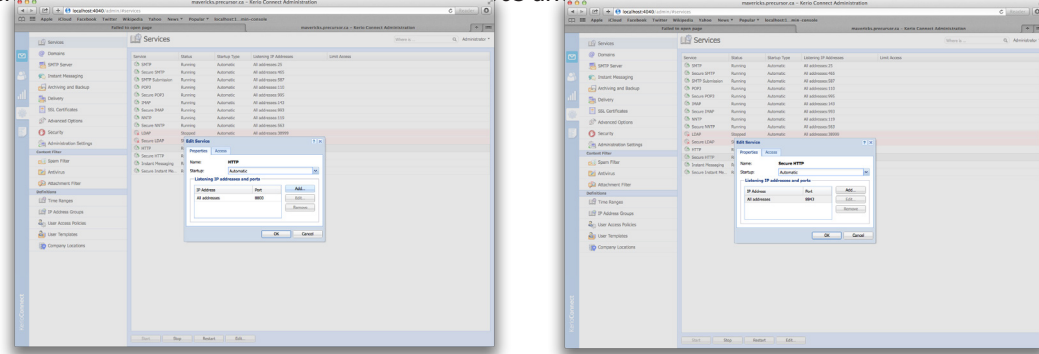


Fig 7 a & b. Turning off default ports of 80, 8800 and 443, 8843 in Kerio Connect

Incidentally, you will also want to edit the ports for LDAP and Secure LDAP from there default values. This will avoid error messages as the Kerio services cannot run on the same ports as the OS X Server LDAP which was activated earlier when we turned on Open Directory. For this we can just add 99 to the end of the number.

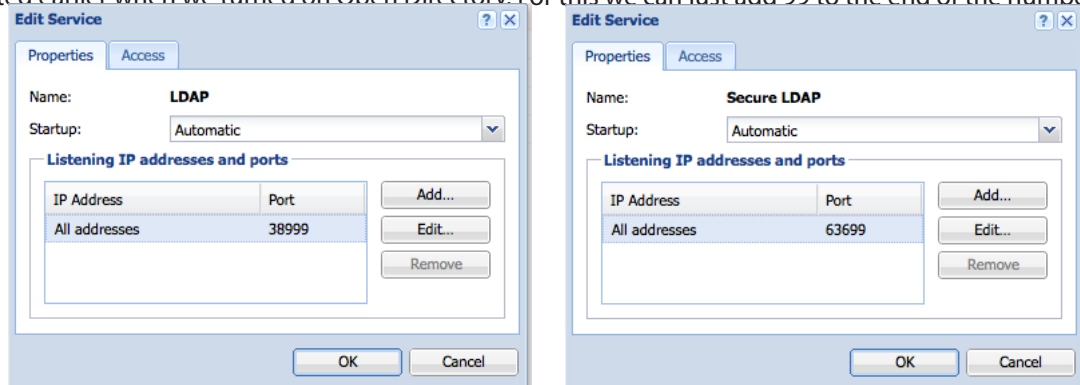


Fig 8 a & b. Editing the LDAP and Secure LDAP ports in Kerio Connect

If you have succeeded you are now ready to set up macOS Server with the reverse proxies.

The MacOS Server Web process has a GUI but you need to do a little customization work to get your Reverse Proxies to show in that GUI.

So before we start the web process lets do the nitty gritty editing. We are going to make Apache web apps. It is not something well documented on Apple's web site or even developer.apple.com although you can read the man pages in Terminal for *webappctl* and *webapp.plist*.

We are going to make some text files for:

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

/Library/Server/Web/Config/apache2/

for example we will need two files for the HTTP of a web-site and two more for the HTTPS or a web site.

Let's start with Rumpus which we assigned HTTP to port 8000 and HTTPS to port 8010 (see figure 2).

For the HTTP port 8000 we will need:

/Library/Server/Web/Config/apache2/httpd_rumpuswebapp.conf

/Library/Server/Web/Config/apache2/ca.precursor.rumpuswebapp.plist

httpd_rumpuswebapp.conf

(I used bold text for emphasis)

```
# from http://httpd.apache.org/docs/2.2/mod/mod_proxy.html#proxypass
RewriteEngine On
RewriteCond %{HTTPS} =off
RewriteRule . - [E=protocol:http,E=port:8000]
RewriteCond %{HTTPS} =on
RewriteRule . - [E=protocol:https,E=port:8010]
ProxyPassReverse / http://rumpus.precursor.ca:8000/
ProxyPass / http://rumpus.precursor.ca:8000/
```

Save a text file with the above contents into /Library/Server/Web/Config/apache2/httpd_rumpuswebapp.conf

Now you are ready for stage 2, the plist.

ca.precursor.rumpuswebapp.plist

Save a text file with the contents below into:

/Library/Server/Web/Config/apache2/webapps/ca.precursor.rumpuswebapp.plist

Contents of plist (Again, I have bolded for emphasis):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">

<!-- See man pages for webapp.plist(5) and webappctl(8) for information about this example webapp.plist -->

<plist version="1.0">
<dict>
    <key>includeFiles</key>
    <array>        <!-- Include files are activated in virtual host when webapp is started -->
        <string>/Library/Server/Web/Config/apache2/httpd_rumpuswebapp.conf</string>
    </array>
    <key>name</key>
    <string>ca.precursor.rumpuswebapp</string>
    <key>displayName</key>        <!-- Name shown in Server app -->
```

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

```

<string>RumpusWebApp</string>
<key>installationIndicatorFilePath</key><!-- The presence of this file indicates
web app is installed -->
<string>/usr/local/Rumpus/rumpus.conf</string>
<key>sslPolicy</key><!-- Determines webapp SSL behavior -->
<integer>0</integer><!-- 0: default, UseSSLWhenEnabled -->
                                <!-- 1:      UseSSLAlways -->
                                <!-- 2:      UseSSLOnlyWhenCertificateIsTrustable -->
                                <!-- 3:      UseSSLNever -->
                                <!-- 4:      UseSSLAndNonSSL -->

</dict>
</plist>

```

The permissions for these files need to be owner: root and group: wheel, 644.

You can easily apply this in Terminal:

```

sudo chown -R root:wheel /Library/Server/Web/Config/apache2/httpd_rumpuswebapp.conf
sudo chmod -R 644 /Library/Server/Web/Config/apache2/httpd_rumpuswebapp.conf
sudo chown -R root:wheel /Library/Server/Web/Config/apache2/webapps/httpd_rumpuswebapp.plist
sudo chmod -R 644 /Library/Server/Web/Config/apache2/webapps/httpd_rumpuswebapp.plist

```

but I like to use the GUI of the utility Batchmod which is available at:
<http://www.lagentesoft.com/Batchmod/>

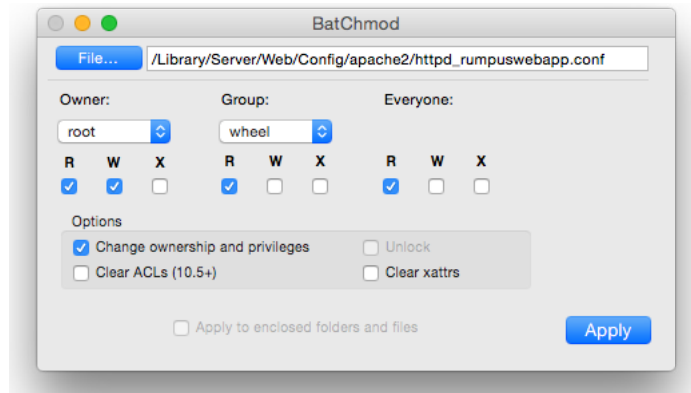


Fig 9a. Using BatchMod to modify permissions

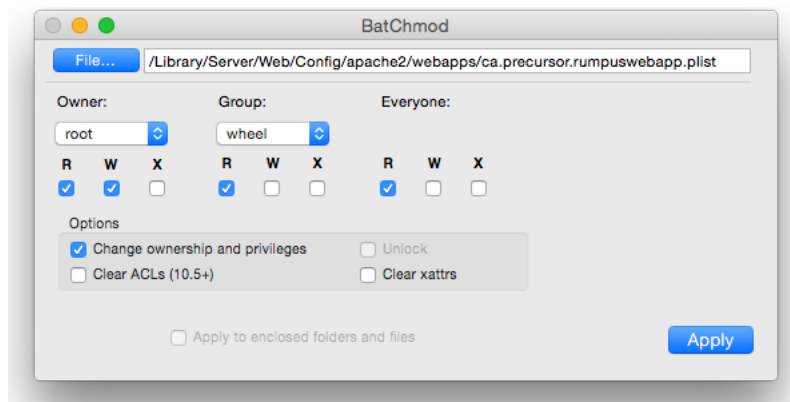


Fig 9b. Using BatchMod to modify permissions

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
 support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

We will make this type of web app file set for each instance of http and https that we need to reverse proxy. And we will have to change the ownership and permissions for each to root:wheel 644. So now lets take a look at the Rumpus SSL files:

httpd_rumpusSSLwebapp.conf

```
# from http://httpd.apache.org/docs/2.2/mod/mod_proxy.html#proxypass
RewriteEngine On

RewriteCond %{HTTPS} =off
RewriteRule . - [E=protocol:http,E=port:8000]
RewriteCond %{HTTPS} =on
RewriteRule . - [E=protocol:https,E=port:8010]
SSLProxyEngine on
ProxyPassReverse / https://rumpus.precursor.ca:8010/
ProxyPass / https://rumpus.precursor.ca:8010/
```

Save a text file with the above contents into `/Library/Server/Web/Config/apache2/httpd_rumpusSSLwebapp.conf` and make sure to change the ownership and permissions accordingly using either Terminal or Batchmod. Note that the bolded line **SSLProxyEngine on** had to be added for Server 5.2.

ca.precursor.rumpusSSLwebapp.plist

Save a text file with the contents below into:

`/Library/Server/Web/Config/apache2/webapps/ca.precursor.rumpusSSLwebapp.plist`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<!-- See man pages for webapp.plist(5) and webappctl(8) for information about this example webapp.plist -->
<plist version="1.0">
<dict>
    <key>includeFiles</key>
    <array>
        <!-- Include files are activated in virtual host when webapp is started -->
        <string>/Library/Server/Web/Config/apache2/httpd_rumpusSSLwebapp.conf</string>
    </array>
    <key>name</key>
    <string>ca.precursor.rumpusSSLwebapp</string>
    <key>displayName</key>
    <string>RumpusSSLWebApp</string>
    <key>installationIndicatorFilePath</key>
    <!-- The presence of this file indicates web app is installed -->
```

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

```

<string>/usr/local/Rumpus/rumpus.conf</string>
<key>sslPolicy</key>      <!-- Determines webapp SSL behavior -->
<integer>0</integer>      <!-- 0: default, UseSSLWhenEnabled -->
                           <!-- 1:      UseSSLAlways -->
                           <!-- 2:      UseSSLOnlyWhenCertificateIsTrustable -->
                           <!-- 3:      UseSSLNever -->
                           <!-- 4:      UseSSLAndNonSSL -->

</dict>
</plist>

```

The Kerio and FileMaker files are going to be similar except you will use the ports and DNS names outlined in figure 4 above.

(NB. see the included example files for FileMaker Server console proxies and note especially that the example conf file provided is now for FileMaker Server 15. When using FMS 12-14 you will need to uncomment a few lines left in them)

For the FileMaker Server <String> use:

```
/Library/FileMaker Server/HTTPServer/conf/httpd.conf
```

And for the config <string> for Kerio you will use its default location:

```
/usr/local/kerio/mailserver/mailserver.cfg
```

If your Kerio mail store has been moved to an external array then of course you would modify as necessary.

To make it easier (i.e. to cut and paste) a complete set of these files is included as a resource in the download of this tutorial at the rais.precursor.ca site. Just substitute your domain wherever you see "precursor.ca" or "ca.precursor" as the case may be. Once you have created and saved all your config and plist files and set their ownership and permissions correctly, restart your computer so that they will get picked up in the GUI in the next step.

SSL Certificates

This is a good time to note that in the next step we are going to configure the custom web sites. In Server 5.2 you must have commercial SSL certificates to enable customweb sites on port 443; self-signed certs are a no go.

So if you haven't already done it, generate the CSR in Server app and purchase and install your SSL Certificates into Server 5.

I recommend installing the certs using Server app and then installing them into your services (Rumpus, FileMaker Server and Kerio Connect).

See the Appendix for more info on exporting and importing certificates.

And don't forget to install the Intermediate Certificate for your commercial authority.

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Enable Web Sites and use the GUI to start the reverse proxies!

Lets start with Rumpus again. First, enable the Web service in macOS Server.

Then create your Rumpus web site: `rumpus.precursor.ca` and leave it set to port 80.

Click the *Edit Advanced Settings...* button:

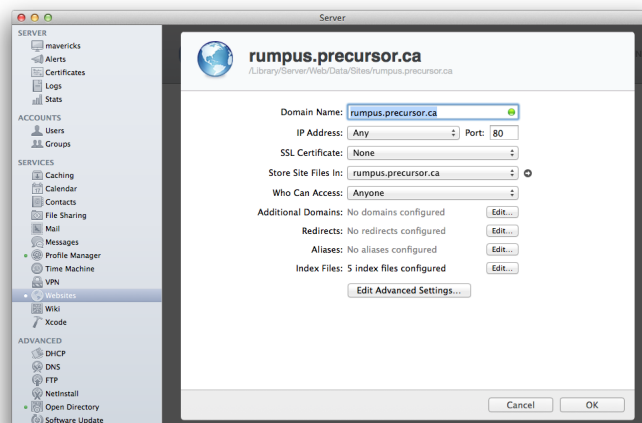


Fig 10. The Advanced Settings button

Now enable the WebApp by putting a checkbox beside *RumpusWebApp*

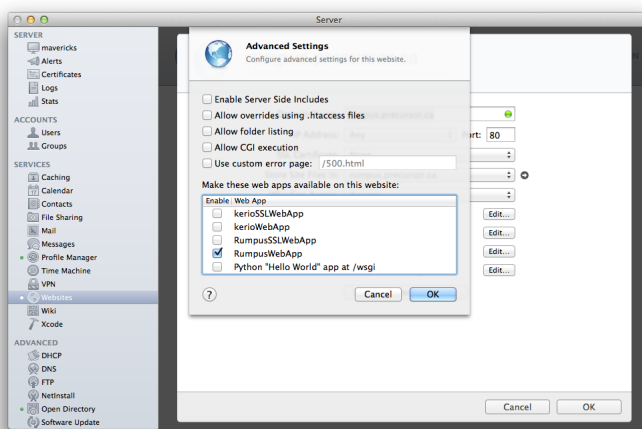


Fig 11. Enable the RumpusWebApp

You will now do the same thing for the HTTP Port 80 sites for FileMaker Server and Kerio Connect

And you will also create the SSL web site for each of these.

Starting with Rumpus again:

Create your Rumpus SSL web site `<rumpus.precursor.ca>` but set it to port 443 and in the Advanced Settings use the *rumpusSSLwebapp*.

Do the same thing for the Kerio SSL and FileMaker Server SSL sites.

There is only one issue to speak of and that is dealing with the Kerio proxy timeout issue:

When Kerio Connect 9 is reversed proxied behind Server app 5.1.7 or higher the web page times out after 60 seconds.

To fix this see Appendix D.

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

You should now have macOS Server Web, Wiki, and Profile Manager all properly using ports 80 and 443 and you should have the following reverse proxies in effect:

`http://rumpus.precursor.ca -> http://rumpus.precursor.ca:8000`
`https://rumpus.precursor.ca -> https://rumpus.precursor.ca:8010`

`http://kerio.precursor.ca -> http://kerio.precursor.ca:8003`
`https://kerio.precursor.ca -> https://kerio.precursor.ca:8013`

`http://filemaker.precursor.ca -> http://filemaker.precursor.ca:8001`
`https://filemaker.precursor.ca -> https://filemaker.precursor.ca:8011`

You will also have the following functional URLs:

macOS Server: `http://macOSServer.precursor.ca`
 FileMaker console: `http://localhost:16001/admin-console`
 Kerio Connect Admin: `http://localhost:4040` or `https://kerio.precursor.ca:4040`

And, if you choose to enable these services in macOS Server:

macOS Server Wiki: `http://macOSServer.precursor.ca/wiki`
 macOS Server Profile Manager: `https://macOSServer.precursor.ca/profilemanager`

Additional Resources

Kai Howells of the Australian company Automatica has written an AppleScript to generate the Kerio web app and can be edited to do the others: <https://github.com/kai-h/Kerio-Connect-Webapp-Configurator>

Automatica is at: <http://automatica.com.au/>

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Revision History:

- 2.0.6 March 17/20. Fixed two example files..
- 2.0.5 March 15/20. Added a missing example file.
- 2.0.4 March 14/20. Added information on Reverse Proxy to Synology NAS and fixed various typos in documentation and example files.
- 2.0.3 October 1/17. Tested on macOS High Sierra 10.13/macOS Server 5.4. Added some examples for the one external i.p./ multiple internal computers case. FileMaker Server 16.0.2 WebDirect is now working with reverse proxy!
- 2.0.2 September 24/16. Re-edited for macOS Sierra and macOS Server 5.2. Included "SSLProxyEngine on" for SSL conf files necessitated by changes in macOS Server 5.2. Fixed a problem in FixKerioProxyTimeout.sh for macOS Server 5.2. Added Appendix F: Troubleshooting.
- NB. FileMaker routines do not work - waiting for Sierra compatible FileMaker Server to test again.**
- 2.0.1 August 5/16. Ammended FileMaker web app conf for FileMaker Server 15 compatibility. Updated throughout to refer to Server 5.1.7.
- 2.0.0 June 12/16. Completely revised for El Capitan and Server app 5.1.5.
Includes critial info for dealing with the Kerio Connect proxy timeout issue.
- 1.0.10 October 24/15. Corrected for OS X Server 5.0.15 fixes and revised instructions for v.3.x and v.4.x to be inline with v.5.0.15.
- 1.0.9 October 10/15. Corrected typos in Appendix C: Server 5 and in the kerio example files.
- 1.0.8 October 1/15. Revised with a working solution for the recently Server app 5.
Changed ports for Kerio to 8003 and 8013 to avoid conflicts with Server 5,
Renamed exmple files from "fms13" to "filemaker" for clarity.
Detailed Info on OS X Server 5 appears in newly added Appendix C.
- 1.0.7 July 14/15. Added an Appendix B to cover Reverse Proxy to a second internal server
- 1.0.6 May 31/15. Now includes Instructions for FileMaker Server 14.
- 1.0.5 May 11/15. Corrections in the certificates appendix and corrections of typos in the example files.
- 1.0.4 January 24/15, Important config info about using commercial certs (GoDaddy etc); Fixed typo in httpd_Keriowebapp.conf example file;.
- 1.0.3 December 30/14, Re-edited for Yosemite and Server.app 4. Added file ownership and permission info.
- 1.0.2 September 29/14, Added Appendix A with info on importing/Exporting SSL certificates.
- 1.0.1 July 22/14, Add reference to Automatica's AppleScript
- 1.0 July 19/14, Show how to set up reverse proxies for Rumpus, Kerio Connect, and FileMaker Server 13 on OS Mavericks with Server.app 3.

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Appendix A: Import/Export SSL certificates

It should be obvious that if you are reverse proxying through Apple's web server the service you use behind that must be using the same SSL Security Certificate as the the one you use in macOS Server.

In some cases you may have been using the service (e.g. Kerio Connect) before doing reverse proxy – in which case you will have applied for the certificate within Kerio Connect and need to export it to macOS.

However, if you are starting from scratch it may be easier to apply for, and install your SSL certificate in macOS Server and then export it to the background service (again I will use Kerio as the example).

Exporting from macOS to Kerio Connect or FileMaker Server

Export a .p12 file from Keychain Access

Open Keychain Access

Select "System" and click the Lock icon to unlock it supplying the admin password for your computer.

In Category, select Certificates and find your mail server cert. Select it.

Right Click and choose Export "*kerio.precursor.ca*"...

Choose the File Format : Personal Information Exchange (.p12).

Now, use Terminal to massage the constituent keys you need out of the .p12 file you just exported.

```
openssl pkcs12 -in path.p12 -out kerio.precursor.ca.crt.pem -clcerts -nokeys
-clcerts only outputs client certificates (not CA certificates)
-nokeys no private keys are output
```

```
openssl pkcs12 -in path.p12 -out kerio.precursor.ca.key.pem -nocerts -nodes
-nocerts no certificates at all will be output
-nodes don't encrypt the private keys at all
```

In the Finder you can remove the .pem from the above names.

Finally, import the keys into Kerio Connect by going into the *SSL Certificates* pane and clicking the Import button and select the .key and the .crt files into their appropriate fields.

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Importing into OS X from Kerio Connect

To export you have to be logged in as the root user so you would have to *Enable Root* in Directory Utility from / System/Library/CoreServices/Applications. Once logged in as root,

In Kerio Connect's *SSL Certificates* pane select the Active Certificate

Export the certificate and the private key files using the *Export* button. You should have:

kerio.precursor.ca.crt and *kerio.precursor.ca.key*.

Use the certificate and key files to make a .p12 file which you can import into OS X Keychain access:

```
openssl pkcs12 -export -clcerts -inkey kerio.precursor.ca.key -in kerio.precursor.ca.crt -out kerio.precursor.ca.p12
```

-clcerts only outputs client certificates (not CA certificates).

Enter Export password: I use login password

Then Verify password:

In Keychain Access select the System Keychain and unlock it using the login password.

Select the Certificates category, then

From the File menu select *Import Items...* and navigate to the .p12 file you made above into the destination *Certificates*. Supply the export password you made earlier.

Third party certificates will not work in MacOS Server without following the tech note below...

Notes.

1) For all of the above you will supply the full path the file into terminal. You can get this by typing it out manually or by dragging the file or the enclosing folder into the Terminal Window.

2) In all case you will substitute your own server FQDN for "*kerio.precursor.ca*"

Using 3rd party Commercial Certificates (GoDaddy, Comodo, GeoTrust, etc.)

MacOS Server's Web Services panel won't let you create SSL websites with 3rd party certs properly unless you deal with a Keychain Access control issue.

See this important Apple Knowledgebase article if you are having trouble selecting port 443 when you choose a 3rd party commercial cert in the GUI:

<http://support.apple.com/en-ca/HT203731>

OS X Server: Access Controls might prevent a certificate identity from working with Server services

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Appendix B: 2 Computer Reverse Proxy

The reverse proxy instructions were for one computer housing both a functioning macOS Server and other servers like Kerio Connect, FileMaker Server and Rumpus. But you may want to have your macOS Server on one machine using the port forwards for port 80 and 443 and put other servers like Kerio on other machines and use their default default ports of 80 and 443. Or even have macOS Server front end port 443 and reverse proxy it to a Synology device on your network using its standard SSL port 5001.

Proxy from macOS to Kerio Connect or FileMaker Server on another computer.

Instead of using an FQDN (Fully Qualified Domain Name) like "kerio.precursor.ca" in the Proxy directive you will use the internal address of the second computer where the Kerio Connect server resides (in the example I use 1.2.3.4).

The port 80 web app < httpd_Keriowebapp.conf> will need to include these lines (where the i.p. address of the Kerio server is substituted for "1.2.3.4"):

```
ProxyPreserveHost On
ProxyPassReverse / http://1.2.3.4:80/
ProxyPass / http://1.2.3.4:80/
ServerName kerio.precursor.ca
```

PreserveHost keeps the name in the URL and ServerName tells which name to keep there.

The port 443 web app < httpd_KerioSSLwebapp.conf> will need to include these lines (where the i.p. address of the Kerio server is substituted for "1.2.3.4"):

```
SSLProxyEngine on
SSLProxyVerify none
SSLProxyCheckPeerCN off
SSLProxyCheckPeerName off
SSLProxyCheckPeerExpire off
ProxyPreserveHost On
ProxyPassReverse / https://1.2.3.4:443/
ProxyPass / https://1.2.3.4:443/
ServerName kerio.precursor.ca
```

PreserveHost keeps the name in the URL and ServerName tells which name to keep there.

Turning off ProxyVerify, ProxyCheckPeerCN, ProxyCheckPeerName, and ProxyCheckPeerExpire will prevent the client browser from getting certificate errors.

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Proxy from macOS to a Synology.

Reverse Proxy to a Synology can be useful if you don't want to expose your Synology to the internet on port 5000 or 5001 (SSL) where bots may be targeting Synology units specifically because they use a default port (5000 or 5001). And it will allow the Synology to work seamlessly with firewalls that block ports above 1024 – like in many hotels.

Instead of using an FQDN (Fully Qualified Domain Name) like "synology.precursor.ca" in the Proxy directive you will use the internal address of the Synology device (in the example I use 1.2.3.4).

The port 80 web app < httpd_Synologywebapp.conf > will need to include these lines (where the i.p. address of the Synology NAS is substituted for "1.2.3.4"):

```
ProxyPreserveHost On
ProxyPassReverse / http://1.2.3.4:5000/
ProxyPass / http://1.2.3.4:5000/
ServerName synology.precursor.ca
```

PreserveHost keeps the name in the URL and ServerName tells which name to keep there.

The port 443 web app < httpd_SynologySSLwebapp.conf > will need to include these lines (where the i.p. address of the Synology NAS is substituted for "1.2.3.4"):

```
SSLProxyEngine on
SSLProxyVerify none
SSLProxyCheckPeerCN off
SSLProxyCheckPeerName off
SSLProxyCheckPeerExpire off
ProxyPreserveHost On
ProxyPassReverse / https://1.2.3.4:5001/
ProxyPass / https://1.2.3.4:5001/
ServerName synology.precursor.ca
```

PreserveHost keeps the name in the URL and ServerName tells which name to keep there.

Turning off ProxyVerify, ProxyCheckPeerCN, ProxyCheckPeerName, and ProxyCheckPeerExpire will prevent the client browser from getting certificate errors.

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Appendix C: OS X Server 5

Apple's new macOS Server v.5 brings some fairly big changes in Reverse Proxy activity. And changes kept occurring in various versions of 5.x. Users should check out the ReadMe in /Library/Server/Web/Config/apache2/ReadMe.txt (reprinted here with spelling mistakes intact from v. 5.4):

Special notes about the web proxy architecture in Server application 5.0:

This version of Server application contains a revised architecture for all HTTP-based services. In previous versions there was a single instance of httpd acting as a reverse proxy for Wiki, Profile, and Calendar/Address services, and also acting as the Websites service. With this version, there is a major change: A single instance of httpd runs as a reverse proxy, called the Service Proxy, and several additional instances of httpd run behind that proxy to support specific HTTP-based services, including an instance for the Websites service.

Since the httpd instance for the Websites service is now behind a reverse proxy, or Service Proxy, note the following:

- */usr/sbin/apachectl controls desktop Apache by loading/unloading the org.apache.httpd launchd job; it should not be used when Server app is installed.*
- */Applications/Server.app/Contents/ServerRoot/usr/sbin/server-apachectl controls Server application's Website Apache by enabling/disabling the com.apple.server.httpd launchd job using Server's launchctl manager, serverctl.*
- **Other instances of Apache are typically running to support the proxy but are not accessible by any apachectl command.**
- *The Websites service httpd instance listens for TCP traffic on:*
 - *localhost, on special 34580 and 34543 TCP ports, for default sites and for custom sites configured with standard ports 80 or 443. These are behind the proxy.*
 - *External IP addresses, on TCP ports other than 80 and 443, for custom sites configured with non-standard ports. These are not behind the proxy.*
- *It is only the external Service Proxy httpd instance that listens on TCP ports 80 and 443; it proxies HTTP requests and responses to Websites and other HTTP-based services.*
- *The external Service Proxy httpd instance also listens on TCP ports 8008, 8800, 8443, and 8843 to support the Calendar & Contacts service, when those services are enabled. Attempting to bind these ports in other services will cause a conflict.*
- *Adjustments have been made to the directives in Websites service config files:*
 - *The Listen directives specify 127.0.0.1; only the Service Proxy listens on an external IP address.*
 - *The VirtualHost directives specify the internal address and port, so for the default sites it's 127.0.0.1:34580 (for the default site) and 127.0.0.1:34543 (for the default SSL site).*
 - *SSL Certificates are configured in the Service Proxy config file, not in the site config files. The Service Proxy is the SSL endpoint, and the TCP traffic between the proxy and the localhost Websites httpd is not encrypted. CGIs and other components such as PHP scripts should not expect environment variable HTTPS=ON; they need to inspect the X_FORWARDED_PROTO variable to determine whether their remote client is using SSL.*
- *To allow correct construction of redirects through the proxy, a ServerName directive is present in*

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

virtual host config files. "ServerName default" is present for wild-card default sites, and "ServerName http[s]://vhostname:nnn" is present in custom sites. For the latter, the scheme and port number nnn are the external ones, not the internal ones.

- CustomLog directives that correctly log external IP addresses are in use.
- Directives that rely on remote IP addresses for correct operation, such as Allow/Deny, Require ip, and also CGIs that use the REMOTE_ADDR variable should operate correctly since mod_remoteip is configured by default.
- The Service Proxy keeps its own set of logs:
 - /var/log/apache2/service_proxy_access.log
 - /var/log/apache2/service_proxy_error.log
- The Service Proxy instance of httpd is started via a wrapper script /Applications/Server.app/Contents/ServerRoot/usr/sbin/httpd-server-wrapper. This script extracts information, such as the configured SSL certificates, from certain configuration plist files, and uses it to set environment variables and "-D" define constants that are passed to httpd. The Apache config files for the Service Proxy are parameterized with these "-D" and environment variables.
- Additional service-specific instances of httpd keep their own logs:
 - /var/log/apache2/services/*_log

There are several files related to the configuration of the Service Proxy should not be modified by administrators:

/Applications/Server.app/Contents/ServerRoot/System/Library/LaunchDaemons/com.apple.serviceproxy.plist
 /Applications/Server.app/Contents/ServerRoot/System/Library/LaunchDaemons/com.apple.service.*

/Library/Server/Web/Config/Proxy/*_serviceproxy_*.
 /Library/Server/Web/Config/Proxy/ServiceProxySettings.*

/Library/Server/Web/Config/ProxyServices/*

To restore the default configuration, from Terminal app:

`sudo /Applications/Server.app/Contents/ServerRoot/usr/sbin/serveradmin command web:command=restoreFactorySettings`

Note that the proxy version of apache runs even when you turn off Web Services in the Server app GUI

You can manipulate it in Terminal. Here is the command to restart it:

`sudo /Applications/Server.app/Contents/ServerRoot/usr/sbin/serviceproxycctl restart`

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
 support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Appendix D: Kerio Connect Proxy Timeout

Because of the massive changes to Server app 5 where all custom sites use Reverse Proxy regardless of my custom web apps, users will find that Kerio Connect web page time out with a reverse proxy error after 60 seconds.

After a great deal of trial and error and I was able to take a suggested value of 630 and tchange the proxy timeout to avoid this.

The value has to be entered in two places.

If you have have edited your web apps as I indicated you need to only run the bash script I created: `FixKerioProxyTimeout.sh`

This script is included in the package of example files in the zipped archive accompanying this tutorial.

To run the script, open Terminal, type:

```
Sudo sh /pathto/FixKerioProxyTimeout.sh
```

(You can either fill in the "pathto" part or just drag and drop the file into Terminal).

The script adds ProxyTimeout 630 into:

`/Library/Server/Web/Config/Proxy/apache_serviceproxy.conf`

and your custom site file itself. (Both are necessary)

`/Library/Server/Web/Config/apache2/sites/ 0000_127.0.0.1_34543_kerio.precursor.ca.conf`

Here are some important notes from the script:

```
#!/bin/bash
# I was inspired to write this script after seeing Tom Bridge's Munki-in-a-box script in action
# https://github.com/tbridge/munki-in-a-box
# The ProxyTimeout value of 630 was suggested to try by Kerio Support
# The ProxyTimeout entry into the Custom site file was suggested by BunnyFu on Apple's support forum
# https://discussions.apple.com/thread/7331447
# The method of putting ProxyTimeout into the apache proxy was suggested by Dr Greg Mulhauser
# https://codedmemes.com/lib/adjust-apache-proxy/
# This method is my attempt to fix a problem via-a-vis Kerio Connect and MacOS Server that neither company is currently fixing
# No warrantee is offered, either express or implied.
# Use at your own risk.
# Alex Narvey | Precursor.ca
```

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Appendix E: Ealier Versions

Rather than rewrite this tutorial to make sense for every version of macOS and macOS Server I took the easy way out and just updated it for macOS High Sierra 10.13.0 and macOS Server 5.4.

But Reverse Proxy works on earlier versions of the OS (prior to 10.11) and if you are using them then the original Tutorial (version 1.1.10) is your best bet.
It is available in the zipped archive of this Tutorial.

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Appendix F: Troubleshooting

From time to time I get emails from users who can't make this work.

In virtually every case the problem was either poor DNS or a typographical error that occurred when modifying the config files for local use.

1) Check and doublecheck your DNS. If your DNS is provided by someone else then you can almost take it for granted that this is your starting point.

2) Check, doublecheck and triplecheck any editing of the conf and plist files you have made. Edit only copies of my originals so you have something to compare to. A handy tool for finding the difference between your version of the file and mine is the `opendiff` tool as discussed by Charles Edge at <http://krypted.com/mac-os-x/47562/>

```
opendiff /path/to/file1 /path/to/file2
```

Of course, this admonition is the pot calling the kettle black since I am the worst proof reader in the world!

3) The log can be very helpful in sorting this out but make sure you are looking at the proxy logs because all custom web sites are now served by the proxy apache: So it will usually be more important to check:

```
/var/log/apache2/service_proxy_error.log
```

than:

```
/var/log/apache2/error.log
```

4) To test modifications you can restart the apache proxy server with this command:

```
sudo /Applications/Server.app/Contents/ServerRoot/usr/sbin/serviceproxysql restart
```

5) If all else fails you can always restore web services to its default config and start again with this command:

```
sudo /Applications/Server.app/Contents/ServerRoot/usr/sbin/serveradmin command web:
command=restoreFactorySettings
```

Precursor Systems

A certified member of the Apple Consultants Network

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880